

High Availability, Networking and Over-the-Air (OTA) Deployment

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

Resilience, Service Modes, and Networking in Swarm

Service Deployment Modes

Replicated vs. Global Services

Modern clusters run diverse workloads: while application services require elastic scaling via **dynamic replicas** based on demand, infrastructure components demand a uniform presence through a **mandatory agent per node**.

- **Replicated Mode (Task-Centric)**
 - ▶ Declares an exact target number of instances.
 - ▶ The Swarm orchestrator schedules tasks based on available capacity.
- **Global Mode (Node-Centric)**
 - ▶ Guarantees exactly one task per eligible node in the cluster.
 - ▶ Scales automatically as new nodes join or leave the swarm.

Replicated Mode

A deployment strategy where you specify the precise number of identical task instances (replicas) to run across the cluster.

- **Key Characteristics**

- ▶ Flexible scaling up or down on demand.
- ▶ High availability through dynamic rescheduling if a node fails.

- **Typical Use Cases**

- ▶ Stateless web applications and frontends.
- ▶ Scalable microservices and REST APIs.
- ▶ Background worker queues handling asynchronous jobs.

Replicated Mode in Practice

Deploying a Replicated Service: Use the `--replicas` flag (or rely on the default behavior) to specify the initial task count:

```
$ docker service create \  
  --name web-api \  
  --replicas 3 \  
  --publish 8080:80 \  
  nginx:alpine
```

Dynamic Scaling on Demand: Scale up or down instantly without service downtime:

```
$ docker service scale web-api=5
```

Global Mode

A deployment strategy where Docker Swarm runs exactly one task instance on every active node in the cluster.

- **Key Characteristics**

- ▶ No manual replica count required.
- ▶ Automatically provisions tasks onto newly added worker nodes.

- **Typical Use Cases**

- ▶ **Monitoring agents:** e.g., Prometheus Node Exporter.
- ▶ **Log collectors:** e.g., Fluentd, Logstash, or Vector.
- ▶ **Local proxies & security daemons:** Node-level network filters or storage drivers.

Global Mode in Practice

Deploying a Global Service: Pass `-mode global` to enforce running one task per node:

```
$ docker service create \  
  --name node-exporter \  
  --mode global \  
  --mount type=bind,source=/proc,target=/host/proc \  
  prom/node-exporter
```

Running `docker service scale` on a global service will return an error, as the instance count is strictly tied to cluster membership.

Connecting Workloads and Networking

The Distributed Challenge

Whether scaling dynamically (**Replicated**) or running mandatory agents (**Global**), container tasks end up isolated across physically separated hosts.

To enable seamless communication between these distributed tasks, we need a robust networking abstraction that transcends the underlying physical infrastructure.

- **Inter-Task Communication Requirements**

- ▶ Replicated microservices must reach each other regardless of where tasks are scheduled.
- ▶ Global infrastructure agents need to securely push data to centralized collectors.

- **The Next Step: Virtualizing the Network**

- ▶ We need a flat layer-2 abstraction over the physical infrastructure: **The Overlay Network.**

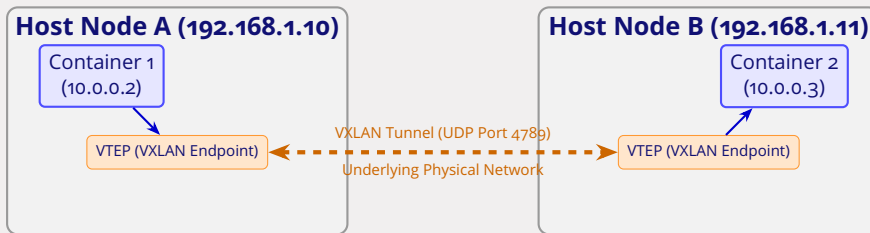
Multi-Host Networking

The Overlay Network

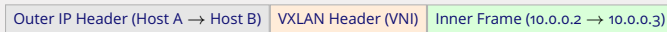
- **The Challenge:** How to enable seamless container-to-container communication across physically separated host nodes without exposing internal ports to the underlying physical network.
- **The Solution: Overlay Networks**
 - ▶ A virtual layer-2 network built on top of the host cluster infrastructure.
 - ▶ Powered by **VXLAN (Virtual Extensible LAN)** encapsulation.
 - ▶ Encapsulates Layer 2 Ethernet frames inside Layer 4 UDP packets.
 - ▶ Treats containers on different hosts as if they were attached to the same local switch.

Overlay Network Architecture

VXLAN Encapsulation and Multi-Host Communication



Encapsulated Packet Structure in Transit



Overlay Networks

Isolation, Security, and Service Discovery

- **Encrypted Data Plane**

- ▶ Optional IPsec tunnel encryption at the overlay network level.
- ▶ Protects traffic in transit between nodes with minimal configuration overhead.

- **Network Isolation**

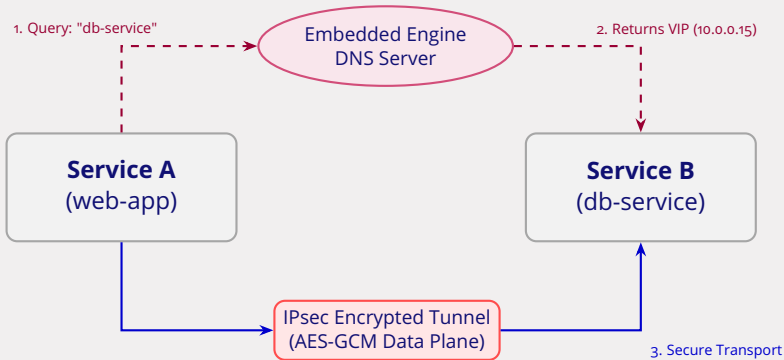
- ▶ Services only communicate if attached to the same overlay network.

- **Embedded DNS Service Discovery**

- ▶ Docker runs an internal DNS server on each daemon.
- ▶ Service names resolve automatically to a Virtual IP (VIP) or container IPs.
- ▶ Provides seamless discovery without external tools (e.g., Consul or Etcd).

Overlay Security & Service Discovery

Internal DNS Resolution and Encrypted Data Plane



Isolation Guarantee: Containers not attached to this overlay network cannot resolve DNS names or access the IPsec tunnel.

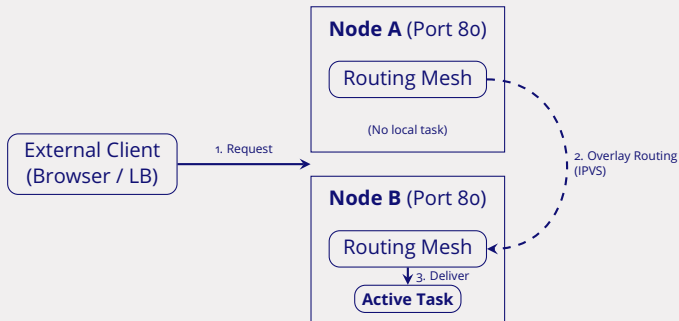
The Routing Mesh

Enabling External Access via Ingress Routing

- **Problem:** External clients need to access services without knowing which specific node hosts the active container tasks.
- **Solution:** Every node accepts incoming traffic on published ports, regardless of task placement. The receiving node acts as an internal **Load Balancer** (via Linux IPVS):
 - ▶ Selects a target healthy task using a round-robin strategy.
 - ▶ Routes the request via direct *unicast* over the Overlay network (no broadcast).
- **Key Advantages**
 - ▶ **Topology Invisibility:** External clients do not need to know which host runs a specific task.
 - ▶ **Simplified Load Balancing:** Allows pointing an external load balancer (e.g., HAProxy, NGINX) to any or all swarm nodes.

Routing Mesh Architecture

Visualising External Ingress Routing



The client hits **Node A**, but the Routing Mesh transparently forwards traffic over the overlay network to the active task running on **Node B**.

Routing Mesh Architecture

Kernel-Level Traffic Distribution with IPVS

- **Ingress Overlay Network**

- ▶ A specialized, multi-host network created automatically by Docker Swarm for external ingress.
- ▶ Connects all cluster nodes to manage incoming traffic on published service ports.

- **Virtual IP (VIP) Allocation**

- ▶ Every service receives a single internal Virtual IP assigned within the overlay network.
- ▶ Internal DNS resolves the service name directly to this VIP.

- **Kernel-Level Forwarding via IPVS**

- ▶ Leverages Linux **IPVS** (*IP Virtual Server*) inside the kernel for transport-layer load balancing.
- ▶ Performs low-overhead, point-to-point Layer-4 packet routing from the ingress node to the destination task.

Service Publishing Modes

Comparing Ingress Mesh and Host Bypass Modes

- **Ingress Mode (Default)**

- ▶ **Mechanism:** Uses the Routing Mesh across all nodes in the cluster.
- ▶ **Behaviour:** Port is exposed on every node; traffic is load-balanced to healthy tasks across the overlay network.
- ▶ **Best For:** Public-facing web apps, APIs, and microservices requiring high availability and topology abstraction.

- **Host Mode (Bypass Mesh)**

- ▶ **Mechanism:** Binds the service port directly to the specific host node running the task.
- ▶ **Behaviour:** Bypasses IPVS and the Ingress Mesh; only nodes running an active container accept traffic on that port.
- ▶ **Best For:** Custom ingress controllers (e.g., Traefik, NGINX Ingress), low-latency requirements, or binding 1:1 host ports.

Remote Software Updates (OTA) and Rolling Updates

Remote Software Updates (OTA) and Rolling Updates

Modernising Production Software Delivery

- **The Evolution of Software Deployment**

- ▶ Traditional deployments relied on scheduled maintenance windows and planned system downtime.
- ▶ Modern distributed applications require **continuous release cycles** without disrupting end users.

- **Why Remote and OTA Updates Matter**

- ▶ Enables rapid delivery of bug fixes, security patches, and new features across cloud and edge clusters.
- ▶ Removes the need for physical access or manual, per-node software installations.

- **Main Challenge in Production:** How to push software updates to live systems reliably, ensuring continuous service availability even when updates encounter runtime failures.

The Remote / OTA Deployment Challenge

Operational Requirements for Distributed Systems

- **Over-The-Air (OTA) Deployment:** Updating software remotely across distributed nodes, edge devices, or cloud clusters without physical access.
- **Core Operational Requirements**
 - ▶ **Zero Downtime:** Applications must maintain availability throughout the update process.
 - ▶ **Rollback Capability:** Immediate recovery mechanism if the new version displays bugs or instability.
 - ▶ **Low Operational Overhead:** Fully automated deployment pipeline requiring zero manual intervention per node.
- **The Distributed Risk:** Network partitions, partial node failures, or unhandled runtime exceptions during updates can cause cluster-wide outages if not managed declaratively.

Continuous Deployment Patterns (1/3)

Deployment Patterns for Zero-Downtime Updates

Rolling Update (Default Swarm Pattern)

- Gradually replaces running task instances with new ones in batches.
- *Trade-off*: Highly resource-efficient; requires backward compatibility between coexisting versions.

Continuous Deployment Patterns (2/3)

Deployment Patterns for Zero-Downtime Updates

Blue/Green Deployment

- Runs two identical environments; router switches 100% of traffic instantly from Blue (old) to Green (new).
- *Trade-off*: Zero version overlap; requires double the hardware capacity (2× resources).

Continuous Deployment Patterns (3/3)

Deployment Patterns for Zero-Downtime Updates

Canary Deployment

- Routes a small fraction of real traffic (e.g., 5%) to the new version before full rollout.
- *Trade-off*: Minimises failure blast radius; requires sophisticated ingress/traffic-splitting proxies.

Configuring Rolling Updates in Swarm

- **Configuration File Context**

- ▶ Rolling update policies are declared within the Docker Compose file (typically `docker-compose.yml` or `stack.yml`) using Compose Specification v3.x.
- ▶ Configured under the `deploy.update_config` block for each individual service.

- **Imperative Management via CLI**

- ▶ Policies can also be applied dynamically via the Docker CLI:

```
docker service update --update-parallelism 2 --update-delay 10s <SERVICE>
```

- **Primary Objective**

- ▶ Provides declarative control over execution speed, safety checks, and failure recovery during automated service deployments.

Rolling Update Parameters

- **parallelism:** Number of tasks updated concurrently (default: 1).
- **delay:** Wait duration between updating consecutive batches of tasks (e.g., 10s, 1m).
- **failure_action:** Action upon failure (pause, continue, rollback).
- **monitor:** Time window after each task update to monitor for container crashes or healthcheck failures (default: 0s).
- **max_failure_ratio:** Tolerance threshold of failed tasks during an update before triggering a failure action.
- **order:** Determines the sequence of task replacement:
 - ▶ **stop-first (Default):** Stops the old task before launching the new one. Lower resource consumption, but brief capacity dip.
 - ▶ **start-first:** Starts the new task first and waits for it to become healthy before stopping the old one. Ensures zero-downtime, but requires spare cluster capacity.

Rolling Update Configuration Example

Complete Production Spec in Compose YAML

Parameter Breakdown

- Updates **2 tasks at a time**.
- Waits **10 seconds** between batches.
- Spawns new tasks **before stopping old ones** (start-first).
- Monitors health for **15 seconds**.
- **Rolls back automatically** if any task fails health checks.

```
version: '3.8'
services:
  web_api:
    image: myrepo/api:v2.0
    deploy:
      replicas: 6
      update_config:
        parallelism: 2
        delay: 10s
        order: start-first
        monitor: 15s
        failure_action: rollback
        max_failure_ratio: 0.1
```

Failure Detection & Automatic Rollback

- **Container Healthchecks as the Detection Engine**

- ▶ Swarm relies on container-level HEALTHCHECK instructions to verify task readiness.
- ▶ If a task fails its healthcheck during the monitor window, it is flagged as *Unhealthy*.

- **Automated Rollback Workflow**

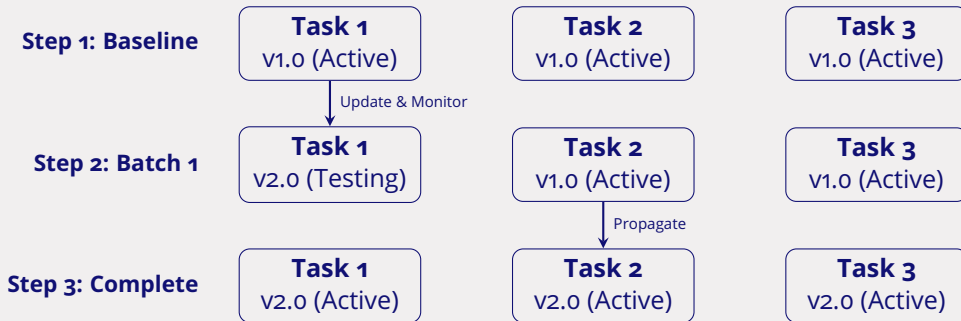
- ▶ When `failure_action: rollback` is configured, Swarm halts the rollout immediately upon detecting an unhealthy batch.
- ▶ Automatically restores running tasks to the previous stable service definition without operator intervention.

Explicit Rollback Execution (CLI)

```
# Manually trigger a service rollback if needed  
docker service update --rollback my_app_web
```

Visualising the Rolling Update Flow

Task Replacement Sequence with Parallelism = 1



Safety Mechanism: If **Task 1 (v2.0)** fails healthchecks during *Step 2*, Swarm halts the rollout and triggers a rollback, leaving **Tasks 2 & 3** undisturbed.

Zero-Downtime OTA: Stateless Architecture

Decoupling Application State for Disposable Tasks

- **The Principle of Ephemeral Containers**
 - ▶ Tasks must be treated as short-lived and disposable units of execution.
 - ▶ No persistent data or session state should be stored on the container's local file system.
- **Externalising Session and Persistent Data**
 - ▶ **User Sessions:** Store in centralized distributed caches (e.g., Redis, Memcached) or use stateless JWT tokens.
 - ▶ **File Storage:** Offload uploads and media to object storage (e.g., AWS S3, MinIO) or shared network volumes (NFS/Ceph).
- **Operational Impact on Rolling Updates**
 - ▶ Enables the Swarm orchestrator to terminate and replace containers instantly without data loss or user session dropouts.

Zero-Downtime OTA: Backward Compatibility

Coexistence of Multiple Versions and Database Schema Evolution

- **Version Coexistence During Rollout**

- ▶ During a rolling update, **v1.0** and **v2.0** containers run concurrently and handle live traffic simultaneously.
- ▶ APIs and internal communication protocols must maintain backward compatibility across adjacent versions.

- **Database Migrations: The Expand-and-Contract Pattern**

- ▶ **Phase 1 (Expand):** Add new columns/tables in a non-breaking manner. Both v1.0 and v2.0 can write to the database.
- ▶ **Phase 2 (Transition):** Deploy v2.0 via Rolling Update while v1.0 phases out.
- ▶ **Phase 3 (Contract):** Once v2.0 is fully deployed and stable, remove legacy columns or deprecated schema elements.

- **Rule of Thumb:** Never execute breaking schema changes in a single deployment step.

Zero-Downtime OTA: Graceful Shutdown

Handling SIGTERM Signals and Connection Draining

- **The Termination Lifecycle**

- ▶ When stopping an old task, Swarm sends a SIGTERM signal to the main process (PID 1).
- ▶ If the container does not exit within a timeout period, Swarm sends a force-kill signal (SIGKILL).

- **Application Responsibility on SIGTERM**

- ▶ Stop accepting new incoming TCP/HTTP requests.
- ▶ Complete all active in-flight requests and background transactions.
- ▶ Close database pools and external socket connections cleanly.

- **Configuring Grace Period in Compose**

```
services:  
  web_api:  
    image: myrepo/api:v2.0  
    stop_grace_period: 30s # Allows up to 30s for graceful drain
```